GETTING STARTED WITH SUITESCRIPT

1. Create and deploy the user event script using Suite script 1.0

Ans.

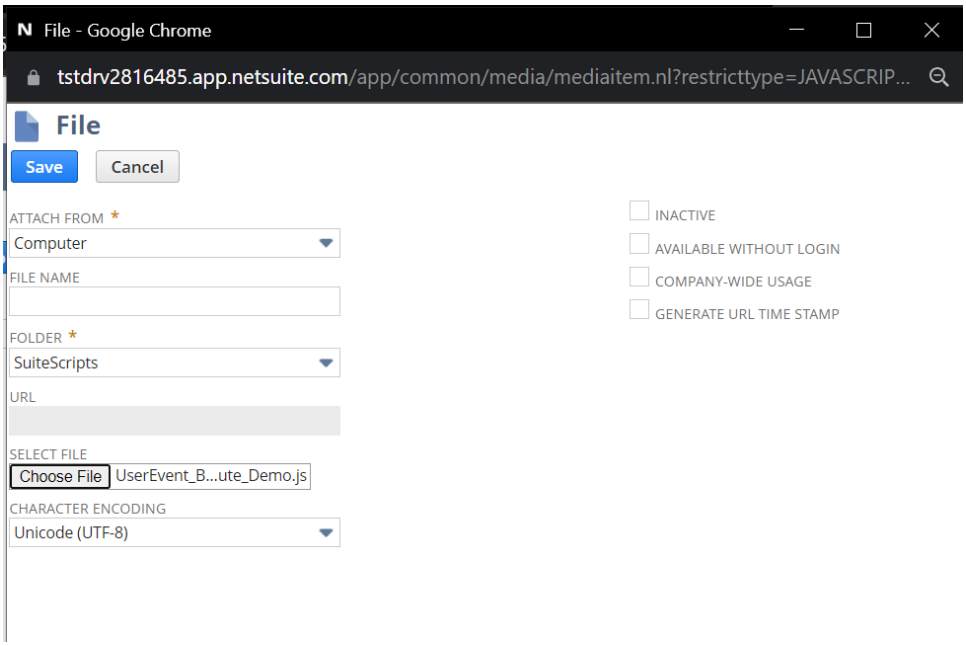
- Create a user event to update a department value of customer record and Pop-up alert after submitting the record.

SS1.js | UserEvent_BFL_SetDepartmentCustomerSillohoute_Demo.js

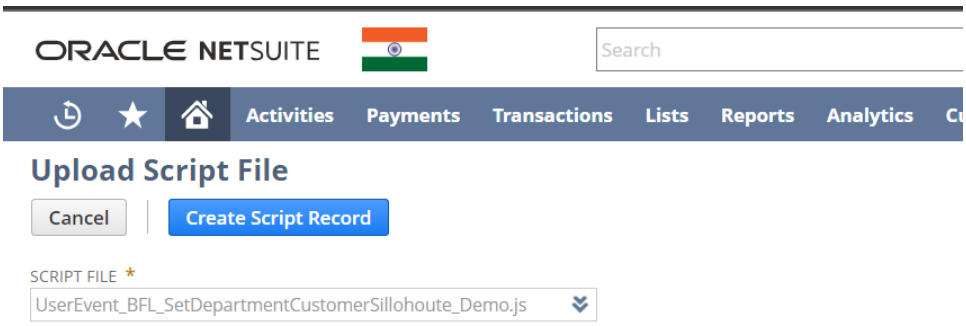
```
function myaftersubmit(){
    var objRecord = nlapiLoadRecord('customer' , 1458);
    objRecord.setFieldValue('custentity_department' , 5 );
    var objGetNameRecord = objRecord.getFieldValue('companyname');
    var objGetSubsidiaryRecord = objRecord.getFieldText('subsidiary');
    var objGetCustomerIdRecord = objRecord.getFieldValue('entityid');
    nlapiLogExecution('debug' , 'myaftersubmit' , objGetCustomerIdRecord +' '+
    objGetNameRecord + 'customer of ' + objGetSubsidiaryRecord + 'subsidiary has been updated');
    alert(objGetCustomerIdRecord +' '+objGetNameRecord + ' customer of ' + objGetSubsidiaryRecord + ' subsidiary has been updated')
    nlapiSubmitRecord(objRecord);

}
```

- Deployment of script: Customization > Scripting > Script > New

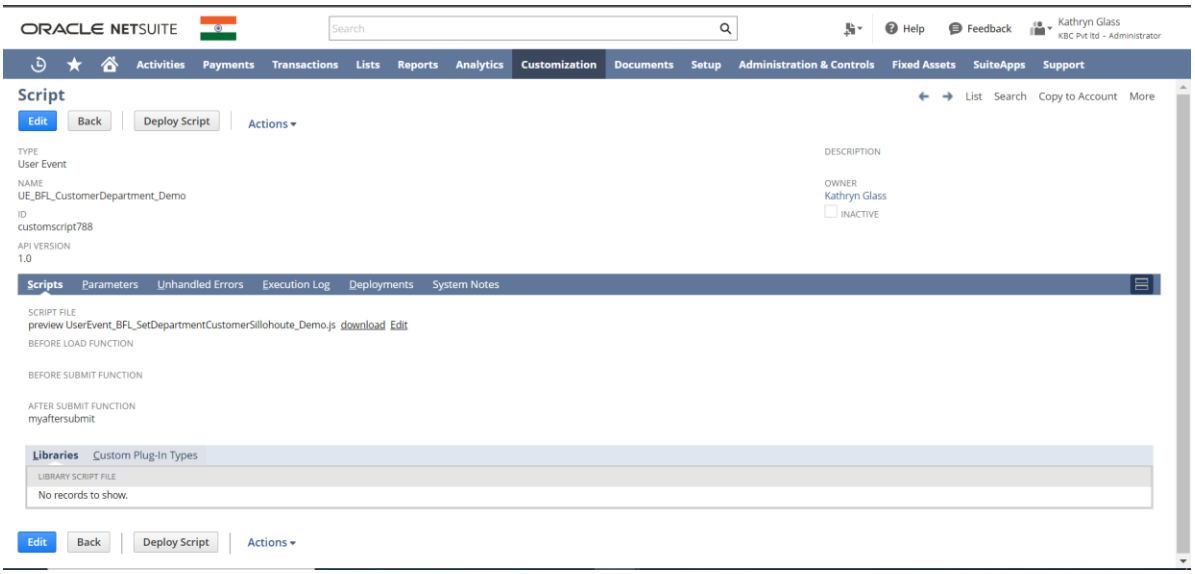
- Upload a new script file:

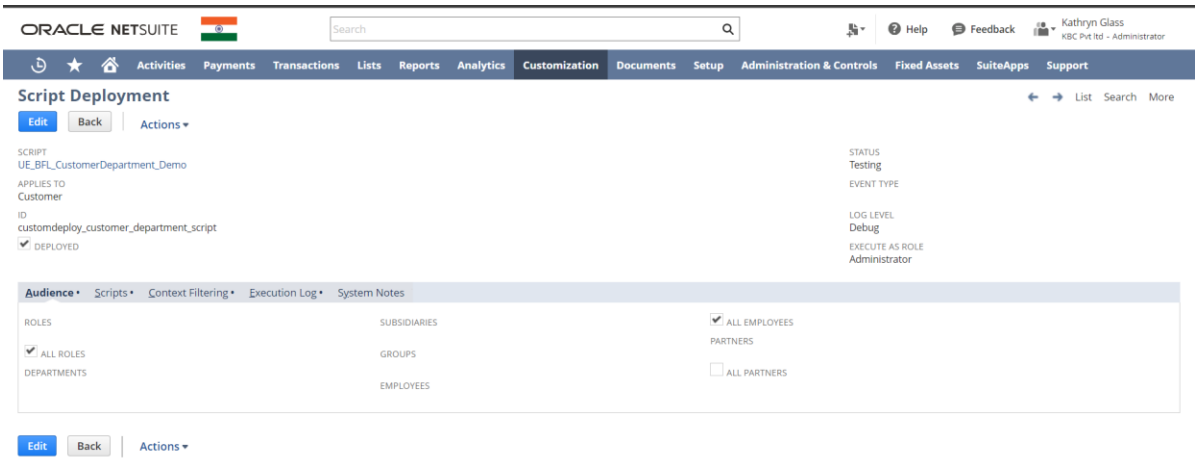


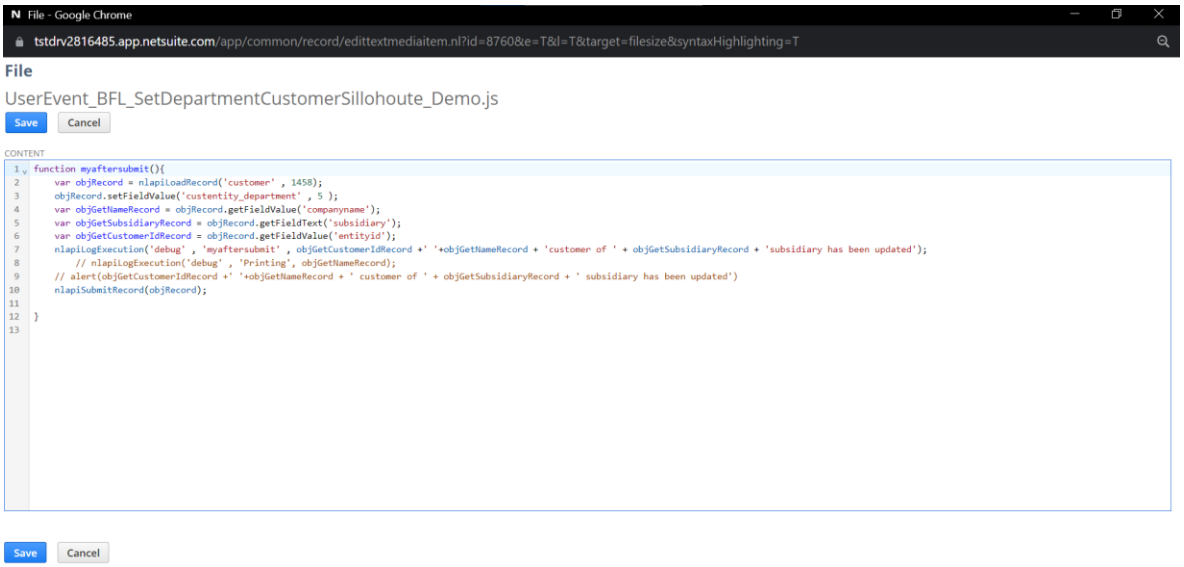
- Create script record > User event



- Deploy the script







- Execution log of script

- Script has been deployed to customer record

2. Create and deploy the user event script using Suite script 2.

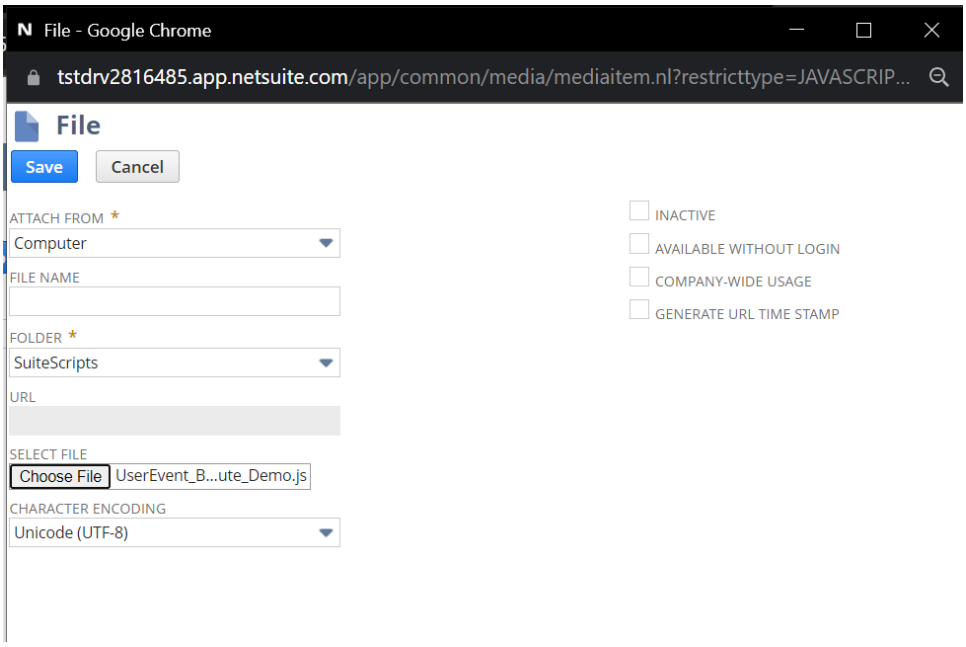
Ans.

- Create a user event to autofill the customer record fields

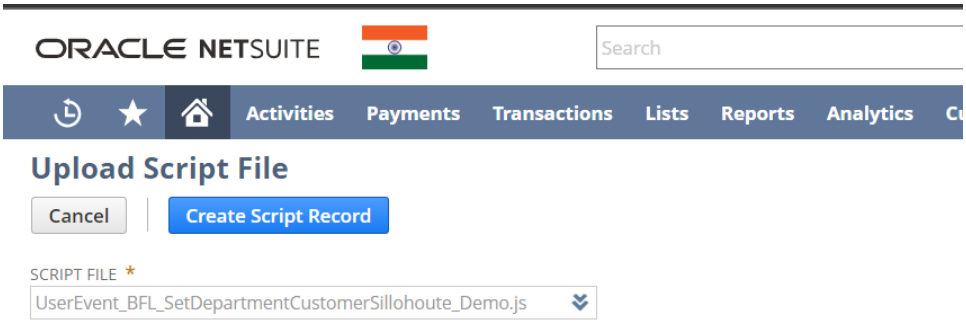
```
/**
 *@NApiVersion 2.x
 *@NScriptType UserEventScript
 */

define(["N/record"], function (record) {

  function mybeforeLoad(context) {
    var objRecord = context.newRecord;
    var t_rec = objRecord.type;
    var i_rec = objRecord.id;
    objRecord.setValue({
      fieldId: "custentity_department",
      value: "1",

      fieldId: "companyname",
      value: "Script Demo Customer",

      fieldId : "entitystatus",
      value: "1",

      fieldId : "subsidiary",
      value: "1",


    });


  }

  function myAfterSubmit(context) {
```

```


  }

  function myAfterSubmit(context) {

    record.submitFields({
      type: t_rec,
      id: i_rec,
      values: {
        custentity_department: 1,
        companyname : "Script Demo Customer",
        entitystatus : 1,
        subsidiary : 1
      },
    });


    log.debug({
    title: "Customer record has been create using script"
    });

  }

  return {
    beforeLoad: mybeforeLoad,
    // beforeSubmit : myBeforeSubmit,
    afterSubmit: myAfterSubmit

  };
});
```

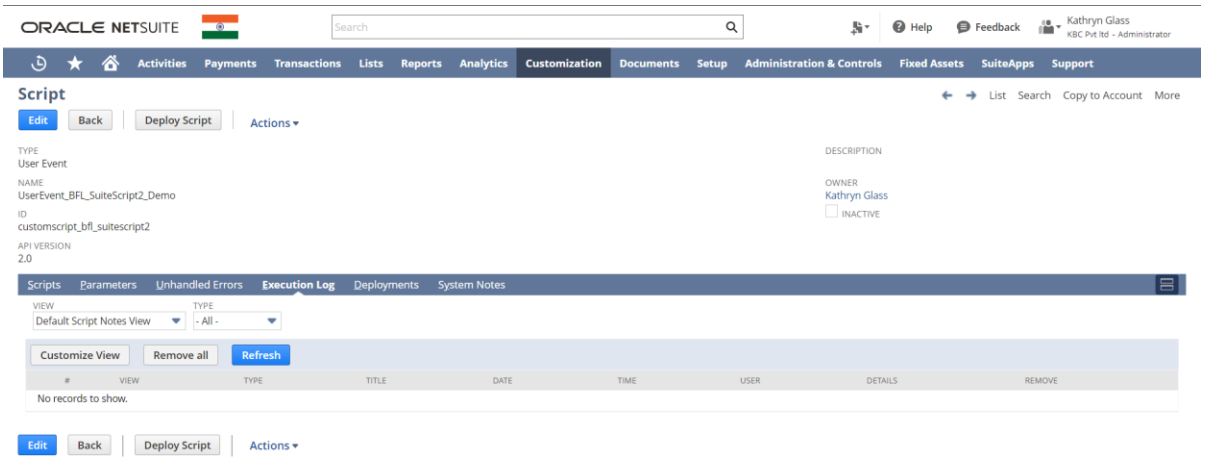
- Deployment of script: Customization > Scripting > Script > New
- Upload a new script file:

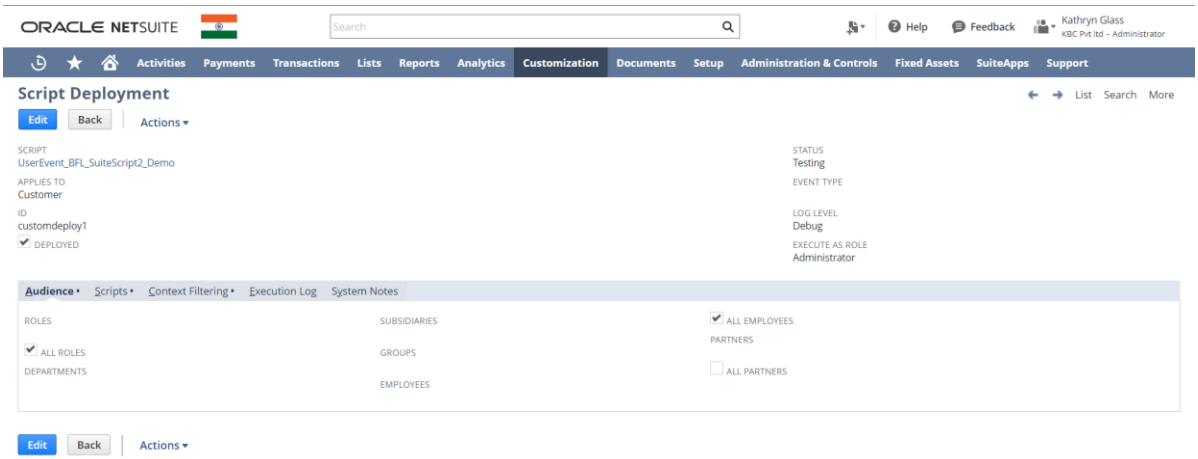


- Create script record > User event

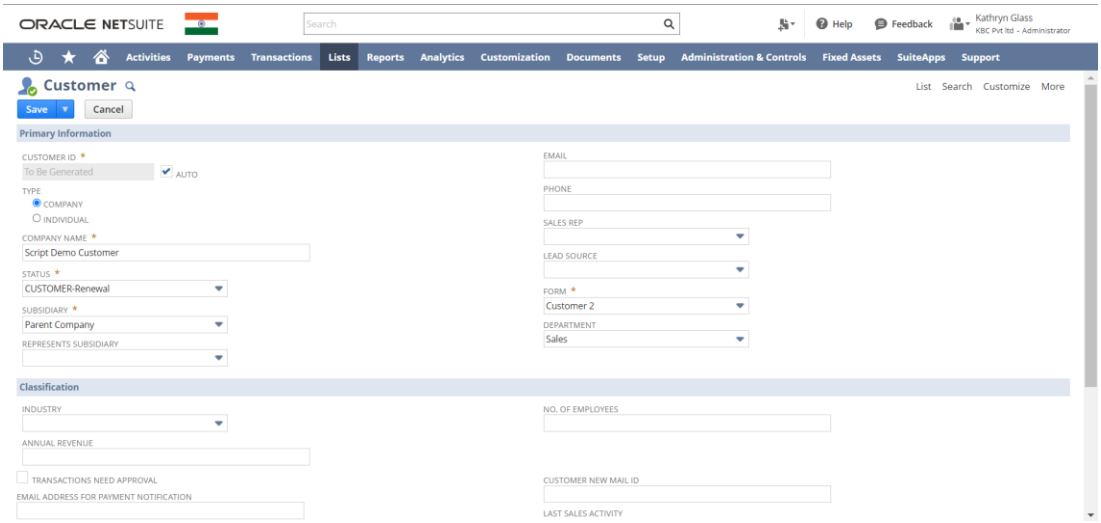


- Deploy the script





- Company name, status and subsidiary has been autofill



3. Write API difference of 1.0 and 2.0 version for below modules. Along with this API details (Min 3 API) 1. Record 2. File 3. Search 4. Submit Field and lookup field (API)

Ans.

Modules	API 1.0	API 2.0

Record	1.Record API start with "nlapi"
	
	2.In 1.0 there is no modular functionality for APIs
	
	3.For Example,
	
	-nlapiCreateRecord(type, initializeValues): this API is used to create a record as we can see we have two parameters for this event i.e., type and parameters.
	
	-type will be used for field ID.
	
	-initializeValues store record value which we want to enter in field.
	
	-In 1.0 sequence of parameters of any API should be same
		1.Record API start with "ModuleName.API({})"
		
		2.API has been categories in modules as per their functionalities, we have separate N/Record module in 2.0
		
		3.For Example,
		- record. Create ({
		type:record.Type.SALES_ORDER,
		isDynamic: true,
		defaultValues: {
		entity: 87
		}
		});
		
		-this API is used to create a new record, here we have three types of parameters.
		
		-Value of "type" key will be record type like customer, SO, PO in which we want to create new record
		
		-isDynamic use to create a record in dynamic mode, this is Boolean record
		
		-if we set true then record will be dynamic and if we set false then record will be non-dynamic
		
		-Again, defaultValues consist of key-value pairs to set default values in record like subsidiary, location etc.
		
		-These parameters are in key: value pair hence, in 2.0 sequence of parameters can be change.

File	1.File API start with "nlapi" 2.In 1.0 there is no modular functionality for APIs 3.For Example, -nlapiCreateFile(name, type, contents): this API is used to create a file here we have three parameters for this event -name consist of file name -type consist of file type -In 1.0 sequence of parameters of any API should be same	1.File API start with "ModuleName.API({})" 2.API has been categories in modules as per their functionalities, we have separate N/File module in 2.0 3.For Example, - var fileObj = file.create({ name: 'test.txt', fileType: file.Type.PLAINTEXT, contents: 'Hello World\nHello World', description: 'This is a plain text file.', encoding: file.Encoding.UTF8, folder: 30, isOnline: true }); -this API is used to create a new file -name consist of file name -fileType consist of file type like "file.Type.PLAINTEXT" -contents contains text or data which we want to store in file and description will contain about purpose of file , In the UI, the value of description displays the Description field on the file record. -encoding type of file will be file.Encoding.UTF8 -folders consist of internal ID of the folder within the NetSuite File Cabinet. -If set to true in isOnline, then users can download the file outside of a current netSuite login session. The default value is false. -These parameters are in key: value pair hence, in 2.0 sequence of parameters can be change.

Search	1.Search API start with "nlapi"
	
	2.In 1.0 there is no modular functionality for APIs
	
	3.For Example,
	
	-nlapiCreateSearch(type, filters, columns): this API is used to create a saved search here we have three parameters for this event
	
	-type consist of record type on which we want to create saved search
	
	-filters consist of conditions for saved search
	
	-mention column will be show in saved search record rest will be unshown
	
	-In 1.0 sequence of parameters of any API should be same
		1.Search API start with "ModuleName.API({})"
		
		2.API has been categories in modules as per their functionalities, we have separate N/search module in 2.0
		
		3.For Example,
		- search. Create ({
		type: search.Type.SALES_ORDER,
		title: 'My Second SalesOrder Search',
		id: 'customsearch_my_second_so_search',
		columns: [{
		name: 'entity'
		}, {
		name: 'subsidiary'
		}, {
		name: 'name'
		}],
		filters: [{
		name: 'mainline',
		operator: 'is',
		values: ['T']
		}],
		});
		-type consist of record type on which we want to create saved search
		-title is label of saved search
		-id will be internal id of search
		-filters consist of conditions in key value pair for saved search
		-columns mention in array will be show in saved search record rest will be unshown

Submit Field and lookup field	1.Search API start with "nlapi"
	
	2.In 1.0 there is no modular functionality for APIs
	
	3.For Example,
	
	-nlapiSubmitField(type, id, fields, values, do Sourcing): this API is used to save any field on database
	
	-type consist of record type in which we have field to submit, id consist of field internal id of NetSuite
	
	-value is record we want to submit
	
	-In 1.0 sequence of parameters of any API should be same
		1.Search API start with "ModuleName.API({})"
		
		2.API has been categories in modules as per their functionalities, we have separate N/record module in 2.0
		
		3.For Example,
		record.submitFields({
		type: record.Type.SALES_ORDER,
		id: 1,
		values: {
		memo: 'ABC'
		},
		options: {
		enableSourcing: false,
		ignoreMandatoryFields : true
		}
		});
		
		-type consist of record type in which we have field to submit, id consist of field internal id of NetSuite
		
		-values consist of key: value pair of field_id : value_of_field
		
		- enableSourcing is used to enable sourcing during the record update. By default, this value is true.
		
		- ignoreMandatoryFields is used to ignore required fields during record submission. By default, this value is false.

4. For invoice, Sales Order, Customer and vendor record get the internal id of:

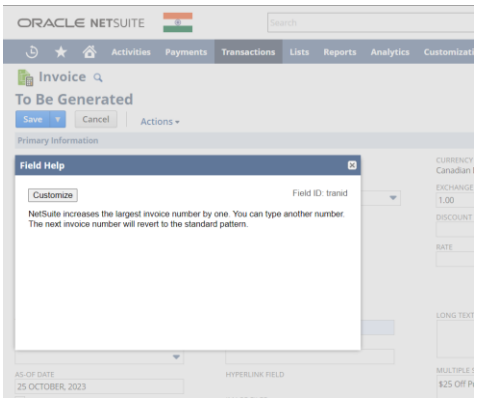
· 5 Body level fields

· 5 Line level fields

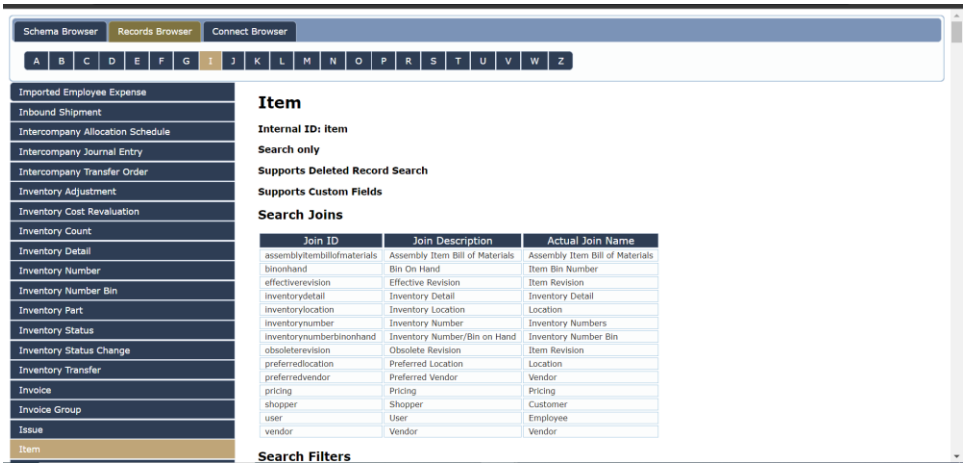
· 3 Sub list IDs

Ans.

- To get internal ids of field: Set Preference > General > Check “show internal id”
- To get an internal id of body field: click on field label > description > Field id



- To get an internal id of line level: Go to Netsuite record browser > Record Browser > Entity/Transaction > Search label



- Invoice:

1.Body field:

Field Label	Field ID
Invoice#	tranid
Customer	entity
Currency	currency
PO#	otherrefnum
Posting Period	postingperiod

2.Line level field:

Field Label	Field ID
Item	item
Description	description
Price level	price
Quantity	quantity
Charge type	chargetype

3. Sub list IDs:

Field Label	Field ID
Item	message
Secondary Book	accountingbook
Exchange Rate	exchangerate
Transaction Is VSOE Bundle	tranisvsoebundle

- Sales Order:

1.Body field:

Field Label	Field ID
Order#	tranid
Sales Rep	salesrep
Date	trandate
Location	location
Discount Item	discountitem
Rate	discountrate

2.Line level field:

Field Label	Field ID
Billing schedule	billingschedule
Taxable	istaxable
Inventory details	inventorydetail
Inventory location	inventorylocation
Inventory subsidiary	inventorysubsidary

3. Sub list IDs:

Field Label	Field ID
Tax Details	taxdetails
Billing address	billingaddress
Shipping address	shipingaddress

- Customer:

1.Body field:

Field Label	Field ID
Customer Id	entityid
Name	altname
Type	isperson
Status	entitystatus
Company Name	companyname
Phone	phone

2.Line level field:

Field Label	Field ID
Primary	isprimary
Balance Amount	balance
Payment method	paymentmethod
Inactive	isinactive
External Id	externalid

3. Sub list IDs:

Field Label	Field ID
Residencial Address	isresidential
Default billing	defaultbilling
Default shipping	defaultshipping

- Vendor:

1.Body field:

Field Label	Field ID
Vendor ID	entityid
Vendor Name	companyname
Print on check as	printoncheckas
Address	defaultaddress
Category	category

2.Line level field:

Field Label	Field ID
Primary	message
Sent email	sendemail
Payment method	paymentmethod
Date	date

3. Sub list IDs:

Field Label	Field ID
Residencial Address	isresidential
Default billing	defaultbilling
Default shipping	defaultshipping

5. On the script deployment record write the purpose of each field mentioned.

Ans:

1.Script: Script use to select script file which we want to deploy

2.Applies to: Select entity/transaction record on which we want to trigger deploying script

3.Id: Script internal custom id

4.Status: Status of script if "Testing" has selected then it will trigger on record of same account and if release has selected then it will be accessible to all user

5.Event type: This is used to trigger script on event. For ex, SO approval, PO rejection.

6.Log level: This is used to print log in Execution log, generally "debug" level use to deploy script, for try-catch block "Error" log level use

7.Execute as role: Here we can select role during deployment, as per role accessibility of record and permission differ. Hence, Administrator always prefer for ease deployment

8.Audience: Here Script can be restricted from employee, department, subsidiary etc.

9.Execution Context: Here we can select where script should trigger like on workflow or on csv import etc.